

重庆邮电大学

CHONGQING UNIVERSITY OF POSTS AND TELECOM.

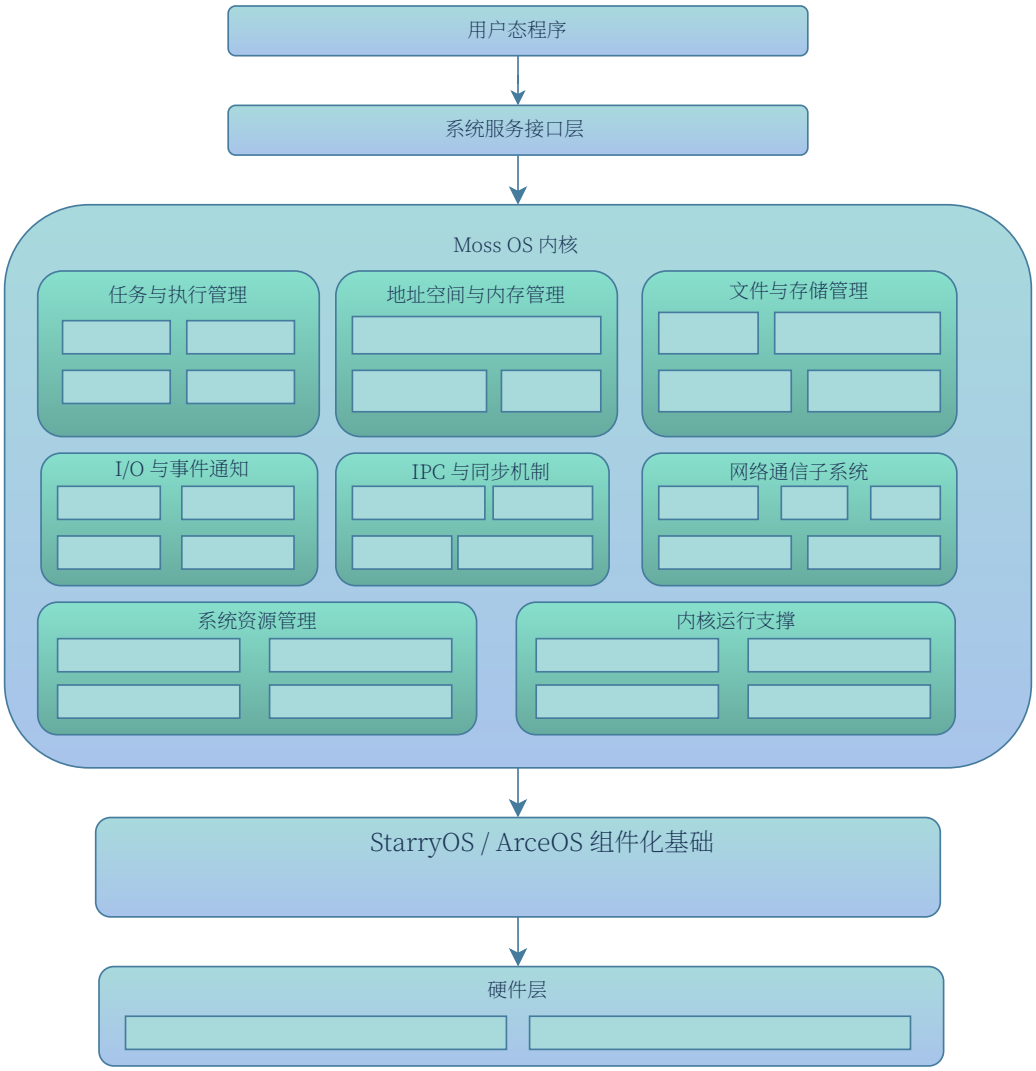
Moss OS

2026 6

1

1.1

1.2


用户态程序
系统服务接口层
Moss OS 内核
任务与执行管理
地址空间与内存管理
文件与存储管理
I/O 与事件通知
IPC 与同步机制
网络通信子系统
系统资源管理
内核运行支撑
StarryOS / ArceOS 组件化基础
硬件层

```
moss
├── Cargo.toml                  # Rust workspace 配置
├── Makefile                    # 项目构建入口
├── src/                        # 用户态入口与初始化脚本
│   ├── main.rs                 # 用户态启动入口配置
│   ├── init.sh                 # 初始化脚本与运行环境准备
│   ├── init_shell.sh           # 交互式 shell 初始化脚本
│   └── ltp_testcases.sh        # 测试用例列表与运行配置
├── kernel/src/                 # moss 内核主体
│   ├── config/                 # 内核配置与运行参数
│   ├── task/                   # 任务、进程、线程、信号与 futex
│   ├── mm/                     # 地址空间、ELF 加载、mmap 与缺页处理
│   │   └── aspace/             # 用户地址空间与映射后端
│   ├── file/                   # 文件对象、FD 表、pipe、eventfd 与 epoll
│   ├── pseudofs/               # procfs、sysfs、devfs 等伪文件系统
│   │   └── dev/                # 设备文件节点与 tty 支持
│   └── syscall/                # Linux 风格系统调用分发与实现
│       ├── task/               # 进程、线程、wait、exec、ptrace 等调用
│       ├── mm/                 # brk、mmap、mremap、mlock 等调用
│       ├── fs/                 # VFS、路径、mount、xattr 与文件 I/O 调用
│       ├── io_mpx/             # select、poll、epoll 等 I/O 多路复用
│       ├── ipc/                # System V IPC 与共享内存相关调用
│       ├── sync/               # futex、membarrier 等同步相关调用
│       └── net/                # socket、网络选项与 BPF 相关调用
├── third_party/                # 第三方组件与外部库适配
│   ├── lwext4_rust/            # Ext4 支持库及 Rust 封装
│   └── syscalls/               # 多架构 syscall 编号与辅助表
├── vendor/                     # 固定版本的本地依赖与 ArceOS 组件
├── make/                       # 构建规则、平台配置与链接脚本
├── scripts/                    # 镜像准备、运行测试与辅助脚本
└── docs/                       # 项目文档与报告材料
```

1.3

	kernel-rv kernel-la

	/proc /sys /dev

sys_*

/proc /sys /dev

2

2.1 StarryOS

2.1.1 StarryOS

2.1.2

`/proc/self/ns/*`

2.2

2.2.1 UID/GID

```rust
pub struct ProcessData {
    // ... 其他字段 ...
    umask: AtomicU32,
    ioprio: AtomicU32,
    uid: AtomicU32,
    euid: AtomicU32,
    suid: AtomicU32,
    gid: AtomicU32,
    egid: AtomicU32,
    sgid: AtomicU32,
}
```

2.2.2 setuid

kernel/src/syscall/task/ctl.rs


```rust
pub fn sys_setuid(uid: u32) -> AxResult<isize> {
    let curr
```

```
    let proc_data = &curr.as_thread().proc_data;
    let old_ruid = proc_data.uid();
    let old_euid = proc_data.euid();
    let old_suid = proc_data.suid();

    // 计算新的 EUID（-1 表示不修改）
    let new_euid = if euid != UID_GID_NO_CHANGE { euid } else { old_euid };

    // 非特权进程权限检查
    if old_euid != 0 {
        if !valid_id_change(ruid, &[old_ruid, old_euid])
            || !valid_id_change(euid, &[old_ruid, old_euid, old_suid])
        {
            return Err(AxError::from(LinuxError::EPERM));
        }
    }

    // 应用修改
    if ruid != UID_GID_NO_CHANGE {
        proc_data.set_real_uid(ruid);
    }
    if euid != UID_GID_NO_CHANGE {
        proc_data.set_euid(euid);
    }

    // Linux 语义：修改 RUID 或将 EUID 设为非 RUID 时，更新 SUID
    if ruid != UID_GID_NO_CHANGE || (euid != UID_GID_NO_CHANGE && euid !=
old_ruid) {
        proc_data.set_suid(new_euid);
    }

    Ok(0)
}
```

2.2.3

has_exec_permission

```
fn has_exec_permission(loc: &Location, uid: u32, gid: u32) -> AxResult<bool> {
    let meta = loc.metadata()?;
    if meta.node_type != NodeType::RegularFile {
        return Ok(false);
    }

    let mode = meta.mode.bits() as u32;

    // root 特权：只要有任意执行位即可
    if uid == 0 {
```

```
        return Ok(mode & 0o111 != 0);
    }

    // 检查所有者执行权限
    if uid == meta.uid {
        return Ok(mode & 0o100 != 0);
    }

    // 检查组执行权限
    if gid == meta.gid {
        return Ok(mode & 0o010 != 0);
    }

    // 检查其他用户执行权限
    Ok(mode & 0o001 != 0)
}
```

2.3

2.3.1

```
#[derive(Debug, Clone, Copy)]
pub enum ProcessWaitStatus {
    Stopped(Signo),    // 进程被停止信号暂停
    Continued(Signo),  // 进程被 SIGCONT 继续
}

pub struct ProcessData {
    // ... 其他字段 ...
    /// 作业控制状态，供 wait 系统调用查询
    pub wait_status: SpinNoIrq<Option<ProcessWaitStatus>>,
    /// 唤醒等待继续的线程
    pub stop_event: Arc<PollSet>,
    // ...
}
```

kernel/src/task/signal.rs

```
SignalOSAction::Stop => {
    // 记录停止状态
    *thr.proc_data.wait_status.lock() =
Some(ProcessWaitStatus::Stopped(signo));
```

```
    // 唤醒父进程（可能正在 wait）
    wake_waiting_parent(&thr.proc_data);
    // 阻塞当前线程，直到收到 SIGCONT
    wait_for_continue(&thr.proc_data);
}
```

```
fn wait_for_continue(proc_data: &ProcessData) {
    block_on(poll_fn(|cx| {
        let stopped = matches!(
            *proc_data.wait_status.lock(),
            Some(ProcessWaitStatus::Stopped(_))
        );
        if !stopped || proc_data.proc.is_zombie() {
            Poll::Ready(())
        } else {
            proc_data.stop_event.register(cx.waker());
            Poll::Pending
        }
    }));
}
```

```
SignalOSAction::Continue => {
    *thr.proc_data.wait_status.lock() =
Some(ProcessWaitStatus::Continued(signo));
    wake_waiting_parent(&thr.proc_data);
    // stop_event.wake() 在其他地方调用
}
```

2.3.2 wait

kernel/src/syscall/task/wait.rs

```
fn wait_status_siginfo(child: &Process, options: WaitOptions)
    -> AxResult<Option<siginfo_t>>
```

```
{
    let Ok(proc_data) = get_process_data(child.pid()) else {
        return Ok(None);
    };

    let mut status = proc_data.wait_status.lock();
    let Some(wait_status) = *status else {
        return Ok(None);
    };

    match wait_status {
        // 检查是否报告停止状态
        ProcessWaitStatus::Stopped(signo) if
options.contains(WaitOptions::WUNTRACED) => {
            // 非破坏性查询：不清除状态
            if !options.contains(WaitOptions::WNOWAIT) {
                *status = None;
                proc_data.stop_event.wake();
            }
            Ok(Some(make_sigchld_info(child.pid(), signo as i32, CLD_STOPPED)))
        }
        // 检查是否报告继续状态
        ProcessWaitStatus::Continued(signo) if
options.contains(WaitOptions::WCONTINUED) => {
            if !options.contains(WaitOptions::WNOWAIT) {
                *status = None;
            }
            Ok(Some(make_sigchld_info(child.pid(), signo as i32,
CLD_CONTINUED)))
        }
        _ => Ok(None),
    }
}
```

```
fn make_exit_siginfo(child: &Process) -> siginfo_t {
    let wait_status = child.exit_code();
    let term_sig = wait_status & 0x7f;

    let (status, code) = if term_sig == 0 {
        (wait_status >> 8, CLD_EXITED)      // 正常退出
    } else if wait_status & 0x80 != 0 {
        (term_sig, CLD_DUMPED)              // 信号终止且 core dump
    } else {
        (term_sig, CLD_KILLED)              // 信号终止
    };

    make_sigchld_info(child.pid(), status, code)
}
```

2.3.3

kernel/src/syscall/task/job.rs

```rust
pub fn sys_setpgid(pid: u32, pgid: u32) -> AxResult<isize> {
    let curr = current();
    let curr_pid = curr.as_thread().proc_data.proc.pid();

    // pid=0 表示当前进程
    let target_pid = if pid == 0 { curr_pid } else { pid };

    // pgid=0 表示使用 target_pid 作为 pgid
    let new_pgid = if pgid == 0 { target_pid } else { pgid };

    // 获取目标进程
    let target_proc = if target_pid == curr_pid {
        curr.as_thread().proc_data.proc.clone()
    } else {
        get_process_data(target_pid)?.proc.clone()
    };

    // 设置进程组
    target_proc.group().set_pgid(new_pgid);
    Ok(0)
}
```

2.4

2.4.1

```rust
#[derive(Debug, Clone, Copy)]
pub struct NamespaceState {
    pub inode: u64,                // 命名空间的唯一标识
    pub parent_inode: Option<u64>, // 父命名空间 inode
    pub init_pid: Option<u32>,     // 命名空间内的 init 进程 PID
}

#[derive(Debug, Clone, Copy)]
pub struct ProcessNamespaces {
    pub pid: NamespaceState,
    pub uts: NamespaceState,
```

```
    pub user: NamespaceState,
}

pub struct ProcessData {
    // ... 其他字段 ...
    namespaces: SpinNoIrq<ProcessNamespaces>,
    // ...
}
```


2.4.2 PID

```
pub fn inherit_namespaces_from(
    &self,
    parent: &ProcessData,
    new_pid: bool,
    new_user: bool,
    new_uts: bool,
) {
    let mut namespaces = parent.namespaces();

    if new_pid {
        // 创建新 PID namespace，当前进程成为 init 进程
        namespaces.pid = NamespaceState::child_of(
            namespaces.pid,
            Some(self.proc.pid())
        );
    }
    if new_user {
        namespaces.user = NamespaceState::child_of(namespaces.user, None);
    }
    if new_uts {
        namespaces.uts = NamespaceState::child_of(namespaces.uts, None);
    }

    *self.namespaces.lock() = namespaces;
}
```

```rust
pub fn visible_pid(&self) -> u32 {
    let namespaces = self.namespaces.lock();
    if namespaces.pid.init_pid == Some(self.proc.pid()) {
        1  // 命名空间 init 进程显示为 PID 1
    } else {
        self.proc.pid()  // 其他进程显示真实 PID
    }
}
```

/proc/self/ns/*

2.5

2.5.1 clone


```
        return Err(AxError::InvalidInput);
    }

    Ok(())
}
```

```
let new_proc_data = if flags.contains(CloneFlags::THREAD) {
    // 线程: 共享 ProcessData
new_task.ctx_mut().set_page_table_root(old_proc_data.aspace.lock().page_table_root());
    old_proc_data.clone()
} else {
    // 进程: fork 新 ProcessData
    let proc = old_proc_data.proc.fork(tid);
    let new_aspace = if flags.contains(CloneFlags::VM) {
        old_proc_data.aspace.clone()  // CLONE_VM: 共享地址空间
    } else {
        Arc::new(Mutex::new(old_proc_data.aspace.lock().fork()))  // COW 复制
    };
    // 创建新 ProcessData...
};
```

2.5.2 execve

```
pub fn sys_execveat(
    uctx: &mut UserContext,
    dirfd: c_int,
    path: *const c_char,
    argv: *const *const c_char,
    envp: *const *const c_char,
    flags: u32,
) -> AxResult<isize> {
    let path = vm_load_string(path)?;
    let exec_loc = if path.is_empty() && (flags & AT_EMPTY_PATH != 0) {
        // AT_EMPTY_PATH: dirfd 直接指向可执行文件
        let fd_table = FD_TABLE.current();
        let file = fd_table.get(dirfd)?.inner.downcast_ref::<File>()?;
```

```
        Some(file.inner().location().clone())
    } else {
        // 普通路径解析
        resolve_at(dirfd, &path, flags).ok()
    };
    execve_common(uctx, path, exec_loc, None, argv, envp)
}
```

#!

```
if content.starts_with(b"#!") {
    let shebang_line = content.split(|&b| b == b'\n').next()?;
    let interpreter = parse_shebang(shebang_line)?;
    // 递归执行解释器，原文件路径作为参数
    return execve_common(uctx, interpreter, None, Some(&path), argv, envp);
}
```

2.6

2.6.1 ptrace


```
pub fn ptrace_wait_if_stopped(thr: &Thread) {
    let pid = thr.proc_data.proc.pid();
    if !ptrace_is_traced(pid) {
```

```
        return;
    }

    // 检查是否有待处理的停止事件
    while ptrace_should_stop(pid) {
        // 阻塞等待调试器发送 PTRACE_CONT 或 PTRACE_SYSCALL
        block_on(ptrace_wait_continue(pid));
    }
}
```

2.6.2 prctl

```
pub fn sys_prctl(option: i32, arg2: usize, arg3: usize, arg4: usize, arg5:
usize)
    -> AxResult<isize>
{
    match option as u32 {
        PR_SET_NAME => {
            // 设置进程名称（最多 16 字节）
            let name = vm_load_until_nul(arg2 as *const u8, 16)?;
            current().set_name(name);
            Ok(0)
        }
        PR_GET_NAME => {
            // 获取进程名称
            let name = current().name();
            let buf = arg2 as *mut u8;
            buf.vm_write_slice(name.as_bytes())?;
            Ok(0)
        }
        PR_SET_PDEATHSIG => {
            // 设置父进程死亡时发送的信号
            // 实现省略
            Ok(0)
        }
        PR_CAPBSET_READ => {
            // 查询 capability bounding set
            let cap = arg2 as u32;
            Ok(if current_has_cap(cap) { 1 } else { 0 })
        }
        _ => {
            warn!("Unsupported prctl option: {}", option);
            Ok(0)  // 返回成功，避免程序崩溃
        }
```

```
    }
}
```

2.6.3 pidfd

```
pub fn sys_pidfd_open(pid: u32, flags: u32) -> AxResult<isize> {
    // 获取目标进程
    let proc_data = get_process_data(pid)?;

    // 创建 PidFd 对象
    let pidfd = PidFd::new(pid, proc_data.exit_event.clone());

    // 加入文件描述符表
    let fd_table = FD_TABLE.current();
    let fd = fd_table.insert(Arc::new(pidfd))?;

    Ok(fd as isize)
}

pub fn sys_pidfd_send_signal(pidfd: i32, sig: u32, info: *mut siginfo_t, flags:
u32)
    -> AxResult<isize>
{
    let fd_table = FD_TABLE.current();
    let pidfd_obj = fd_table.get(pidfd)?.inner.downcast_ref::<PidFd>()?;

    let pid = pidfd_obj.pid();
    let signo = Signo::from_repr(sig as u8).ok_or(AxError::InvalidInput)?;

    // 发送信号到目标进程
    send_signal_to_process(pid, SignalInfo::new_kernel(signo))?;

    Ok(0)
}
```